PSG COLLEGE OF TECHNOLOGY

Coimbatore, Tamil Nadu

DEPARTMENT OF INFORMATION TECHNOLOGY

23I701 - CLOUD AND EDGE COMPUTING

A PROJECT REPORT ON

SMART WATER QUALITY MONITORING AND DIGITAL TWIN

An IoT and Edge-Computing Platform using Docker, MQTT, Node-RED, InfluxDB, Grafana and Eclipse Ditto

Submitted in partial fulfilment of the requirements for the degree of

BACHELOR OF TECHNOLOGY

in

INFORMATION TECHNOLOGY

Submitted by

Giretharan M - 23i218
Vishweshvar S - 23i274
Lakshmandev Vk - 24i433
Santhosh P - 24i435

ABSTRACT

Continuous monitoring of stored water is essential to protect public health, yet manual sampling is slow and provides only occasional snapshots. This project presents **Smart Water Quality Monitoring and Digital Twin**, an IoT-based platform that monitors pH, turbidity, temperature and water level of three water tanks in real time and maintains a live digital twin of every tank.

A Python sensor simulator publishes readings every five seconds over MQTT to an Eclipse Mosquitto broker. Node-RED, acting as the edge-computing layer, validates each reading, applies configurable thresholds and classifies the tank as SAFE, WARNING or UNSAFE. The processed data is written to InfluxDB for historical storage, pushed to Eclipse Ditto to update the digital twin of the tank, and published to an MQTT alert topic when the status is WARNING or UNSAFE. Grafana dashboards, provisioned automatically, display live values and history for each tank. The entire system of twelve containers is deployed with a single Docker Compose command.

Testing with injected abnormal conditions showed that invalid data is rejected at the edge, that unsafe water conditions raise alerts within moments, and that the digital twin and the database stay synchronised with the sensor stream. The architecture is modular and sensor-agnostic, so physical sensors, secure communication and predictive analytics can be added in future.

Keywords: *Internet of Things, Digital Twin, Edge Computing, MQTT, Node-RED, InfluxDB, Grafana, Eclipse Ditto, Docker, Water Quality Monitoring.*

TABLE OF CONTENTS

Abstract i

Table of Contents ii

List of Figures iv

List of Tables v

CHAPTER 1 INTRODUCTION 1

1.1 Background 1

1.2 Problem Statement 1

1.3 Objectives 2

1.4 Scope of the Project 2

1.5 Methodology 2

1.6 Organisation of the Report 3

CHAPTER 2 TECHNOLOGY OVERVIEW 4

2.1 Internet of Things (IoT) 4

2.2 MQTT Protocol 4

2.3 Edge Computing and Node-RED 4

2.4 Time-Series Database: InfluxDB 5

2.5 Digital Twin and Eclipse Ditto 5

2.6 Grafana 5

2.7 Containerisation with Docker 5

2.8 Water-Quality Parameters 5

2.9 Technology Stack Summary 6

CHAPTER 3 SYSTEM ANALYSIS AND DESIGN 7

3.1 Requirement Analysis 7

3.2 System Architecture 8

3.3 Data Flow 9

3.4 MQTT Topic Design and Message Format 9

3.5 Docker Architecture 10

3.6 Project Folder Structure 11

CHAPTER 4 IMPLEMENTATION 12

4.1 Docker Deployment 12

4.2 Python Sensor Simulator 13

4.3 MQTT Broker 15

4.4 Node-RED Edge Processing Layer 16

4.5 InfluxDB Time-Series Storage 19

4.6 Eclipse Ditto Digital Twin 21
4.7 Grafana Dashboards 23
4.8 MQTT Alert System 25
CHAPTER 5 TESTING AND RESULTS 28
5.1 Test Environment 28
5.2 Test Cases 28
5.3 Results and Discussion 29
5.4 Observations 29
5.5 Limitations 29
CHAPTER 6 SECURITY CONSIDERATIONS AND TROUBLESHOOTING 31
6.1 Security Considerations 31
6.2 Troubleshooting 32
CHAPTER 7 FUTURE ENHANCEMENTS 33
CHAPTER 8 CONCLUSION 34
REFERENCES 35
APPENDIX A: SERVICE URLS AND PORTS 36
APPENDIX B: OPERATIONAL COMMANDS 37
APPENDIX C: SOURCE CODE EXTRACTS 38

LIST OF FIGURES

Figure 3.1: Overall system architecture 8
Figure 4.1: Docker Desktop showing the smart-water-monitoring containers and live logs 12
Figure 4.2: Output of docker compose up -d with all 12 containers running 12
Figure 4.3: Container status and ports reported by docker compose ps 13
Figure 4.4: Live telemetry in the simulator log, including an [ABNORMAL] reading 14
Figure 4.5: Simulator injecting a high_temperature scenario into TANK002 15
Figure 4.6: Live sensor messages received by an MQTT subscriber 15
Figure 4.7: Docker Compose logs showing broker, simulator and dashboard activity 16
Figure 4.8: Edge-processing logic of the Node-RED flow 18
Figure 4.9: Node-RED edge flow with the Alert Check function node selected 19
Figure 4.10: Node-RED edge flow with live node status and debug messages 19
Figure 4.11: InfluxDB Data Explorer with water_quality bucket, pH and turbidity fields for TANK002 20
Figure 4.12: InfluxDB container start-up log 21
Figure 4.13: Digital-twin setup script creating the policy and three twins 22
Figure 4.14: Digital twin of TANK001 returned by the Ditto REST API 23
Figure 4.15: Ditto gateway log showing property updates for the twins 23
Figure 4.16: Grafana dashboard for TANK001 24
Figure 4.17: Grafana dashboard for TANK002 24
Figure 4.18: Grafana dashboard for TANK003 25
Figure 4.19: Grafana container log showing Flux queries to InfluxDB 25
Figure 4.20: MQTT alert received on the water/alerts topic 26
Figure 4.21: Mosquitto log showing the alert message on water/alerts 26
Figure 4.22: Node-RED log of alerts published for WARNING and UNSAFE tanks 27

LIST OF TABLES

Table 2.1: Monitored water-quality parameters 6
Table 2.2: Technology stack used in the project 6
Table 3.1: Hardware and software requirements 7
Table 3.2: MQTT topics used by the system 9
Table 3.3: Containers in the Docker Compose deployment 10
Table 4.1: Base profile of each simulated tank 13
Table 4.2: Abnormal scenarios injected by the simulator 14
Table 4.3: Nodes of the Node-RED edge flow 16
Table 4.4: Validation rules applied at the edge 17
Table 4.5: Threshold values used for classification 17
Table 4.6: InfluxDB storage design 20
Table 5.1: Test cases and results 28
Table 5.2: Readings displayed on the Grafana dashboards 29
Table 6.1: Security measures recommended for production 31
Table 6.2: Common problems and their solutions 32
Table 8.1: Service endpoints (local deployment) 36

CHAPTER 1
INTRODUCTION

1.1 Background

Water is the most essential resource for domestic, agricultural and industrial life, and its quality directly affects public health. Storage tanks in residential complexes, institutions and industrial plants are a critical link in the supply chain: even when water leaves the treatment plant in good condition, its quality can degrade inside a tank because of contamination, temperature rise, sediment build-up or changes in chemical balance. Traditionally, tank water quality is checked by manual sampling and laboratory analysis. This approach is slow, labour-intensive, and provides only occasional snapshots, so a problem may go unnoticed for hours or days.

The Internet of Things (IoT) offers a way to monitor water continuously. Sensors measure key parameters such as pH, turbidity, temperature and water level and transmit the readings over a network to software that analyses them in real time. When this analysis is performed close to the data source, a concept called edge computing, abnormal conditions can be detected immediately and bad data can be filtered out before it reaches storage. A further concept, the digital twin, keeps a live virtual replica of every physical asset so that its current state can be queried by any application at any time.

This project, **Smart Water Quality Monitoring and Digital Twin**, combines these ideas into a complete, containerised monitoring platform. A Python simulator generates realistic sensor data for three water tanks. The data travels through an MQTT broker to Node-RED, which acts as the edge-computing layer. The processed results are stored in InfluxDB, mirrored into Eclipse Ditto digital twins, visualised on Grafana dashboards, and published as alerts whenever a tank leaves its safe operating range. The whole system runs as twelve Docker containers started by a single command.

1.2 Problem Statement

Manual water-quality inspection cannot provide continuous visibility, delays the detection of contamination, and leaves no historical record that can be analysed for trends. Existing commercial monitoring solutions are often expensive, closed and difficult to adapt for teaching or small-scale deployments. There is a need for an open, low-cost, modular architecture that can:

- collect water-quality readings from several tanks at regular intervals;
- validate and classify every reading as SAFE, WARNING or UNSAFE at the edge, without waiting for a cloud service;
- store the full history for analysis and display it on live dashboards;
- maintain an always-current digital twin of each tank that other applications can query through a standard REST API; and
- raise real-time alerts when water quality becomes hazardous.

1.3 Objectives

The main objective of the project is to design, implement and test an IoT-based water-quality monitoring system with digital-twin support. The specific objectives are:

1. To simulate pH, turbidity, temperature and water-level sensors for three tanks (TANK001, TANK002 and TANK003) using a Python program.
2. To transport sensor data reliably using the lightweight MQTT publish/subscribe protocol through an Eclipse Mosquitto broker.
3. To implement an edge-computing layer in Node-RED that validates data, applies configurable thresholds and classifies tank status.
4. To store time-series data in InfluxDB and present it on Grafana dashboards with a per-tank selector.
5. To create and continuously update an Eclipse Ditto digital twin for every tank.
6. To generate MQTT alerts for WARNING and UNSAFE conditions.
7. To package the complete solution with Docker Compose so that it can be deployed reproducibly on any machine.

1.4 Scope of the Project

The project covers the full software pipeline from data generation to visualisation and digital-twin synchronisation. Sensor data is simulated rather than read from physical probes, which allows every layer of the system, and the handling of abnormal conditions, to be demonstrated and tested safely and repeatably. The architecture is nevertheless sensor-agnostic: any real device able to publish the same JSON message to the same MQTT topic can replace the simulator without changing the remaining components. The scope does not include hardware design, long-term cloud hosting, or machine-learning-based prediction, which are discussed as future enhancements.

1.5 Methodology

The system was developed in an incremental, layer-by-layer manner. Each layer was first built and tested in isolation and was then integrated with the previous layers:

1. **Requirement analysis:** the monitored parameters, safe ranges, tank count and update frequency were defined.
2. **Architecture design:** the technology stack and data flow between components were chosen (Chapter 3).
3. **Containerisation:** a Docker Compose file was written to describe all services, networks, volumes and health checks.
4. **Implementation:** the simulator, Node-RED flow, InfluxDB schema, Ditto twins and Grafana dashboards were created (Chapter 4).
5. **Testing and validation:** each component and the end-to-end pipeline were tested, and the results were captured as screenshots (Chapter 5).

1.6 Organisation of the Report

Chapter 1 introduces the project. Chapter 2 reviews the technologies and concepts used. Chapter 3 presents the requirements, architecture and design. Chapter 4 describes the implementation of every component with supporting screenshots. Chapter 5 presents testing, results and limitations. Chapter 6 discusses security considerations and troubleshooting. Chapter 7 proposes future enhancements and Chapter 8 concludes the report. References and appendices follow.

CHAPTER 2
TECHNOLOGY OVERVIEW

This chapter introduces the concepts and tools on which the project is built, and explains why each was selected.

2.1 Internet of Things (IoT)

The Internet of Things is a network of physical devices that embed sensors, software and connectivity so that they can collect and exchange data. In water management, IoT devices measure physical and chemical properties and stream the readings to software platforms. Typical benefits include continuous visibility, early warning of contamination, reduced manual effort and the availability of historical data for planning and compliance.

2.2 MQTT Protocol

MQTT (Message Queuing Telemetry Transport) is a lightweight publish/subscribe messaging protocol designed for constrained devices and unreliable networks. Publishers send messages to named topics on a central broker, and subscribers receive every message published on the topics they subscribe to. Because publishers and subscribers never communicate directly, they are fully decoupled, so new consumers can be added without changing the sensors. MQTT offers three quality-of-service levels; this project uses QoS 1 (at least once delivery) for sensor data so that readings are not silently lost. Topic wildcards, such as the single-level wildcard +, allow one subscription to cover all tanks.

Eclipse Mosquitto is an open-source MQTT broker. In this project it listens on port 1883 for MQTT and on port 9001 for WebSocket connections, with persistence enabled so that messages survive a restart.

2.3 Edge Computing and Node-RED

Edge computing means processing data close to where it is produced instead of sending everything unprocessed to a distant cloud. It reduces latency, saves bandwidth, and allows invalid or irrelevant data to be filtered at the source. **Node-RED** is a flow-based, low-code programming tool built on Node.js. Programs are drawn as flows of connected nodes, each performing a small task such as subscribing to MQTT, running a JavaScript function or calling an HTTP API. In this project Node-RED forms the edge layer: it validates each

reading, applies thresholds, classifies the tank status, and forwards the result to InfluxDB, Eclipse Ditto and the alert topic.

2.4 Time-Series Database: InfluxDB

Sensor readings are naturally time-stamped values, so a time-series database is the most efficient way to store them. **InfluxDB 2.7** stores data in *buckets* inside an *organisation*. Each point has a measurement name, indexed tags, fields and a timestamp. Data is written through an HTTP API using the compact line protocol and queried using the Flux language. InfluxDB also provides a web interface (Data Explorer) for ad-hoc queries.

2.5 Digital Twin and Eclipse Ditto

A digital twin is a virtual representation of a physical object that mirrors its state and behaviour. The term was popularised by Michael Grieves in the context of product lifecycle management. **Eclipse Ditto** is an open-source framework for building digital twins. It models every asset as a *Thing* that has attributes (static metadata) and features (dynamic properties), and exposes it through a uniform REST and WebSocket API. Access to every Thing is governed by a *Policy*. Ditto is made up of several microservices (gateway, things, policies, things-search and connectivity) and uses MongoDB for persistence. In this project, a Thing is created for each tank, and its `waterQuality` feature is updated after every reading.

2.6 Grafana

Grafana is an open-source analytics and monitoring platform that turns time-series data into interactive dashboards. It connects to data sources, in this case InfluxDB through Flux queries, and renders the results as stat panels, gauges and time-series graphs. Provisioning files allow the data source and the dashboard to be created automatically at start-up, so no manual configuration is needed after deployment.

2.7 Containerisation with Docker

Docker packages an application and all of its dependencies into an isolated container that behaves identically on any machine. **Docker Compose** describes a multi-container application in a single YAML file, including images, ports, volumes, environment variables, networks, start-up order and health checks. Using Compose, the entire system of twelve containers is started with `docker compose up -d`.

2.8 Water-Quality Parameters

Four parameters are monitored. Their significance, and the reason each is relevant to stored water, are summarised in Table 2.1.

Table 2.1: Monitored water-quality parameters

Parameter	Unit	Significance
pH	pH units (0–14)	Indicates acidity or alkalinity. Drinking water is normally kept between 6.5 and 8.5. Extreme values indicate chemical contamination and can corrode pipes or harm health.
Turbidity	NTU	Measures cloudiness caused by suspended particles. High turbidity can shelter microorganisms and indicates sediment or contamination.
Temperature	°C	Affects dissolved oxygen, chemical reactions and bacterial growth. Warm stored water encourages microbial activity.
Water level	% of capacity	Indicates remaining volume. A very low level signals supply failure, leakage or the risk of dry-running pumps.

2.9 Technology Stack Summary

Table 2.2: Technology stack used in the project

Component	Technology	Version	Purpose
Sensor simulator	Python (paho-mqtt)	3.11 / 1.6.1	Generates readings for three tanks
MQTT broker	Eclipse Mosquitto	2.x	Message transport
Edge layer	Node-RED	Latest image	Validation, thresholds, classification
Time-series database	InfluxDB	2.7	Historical data storage
Dashboard	Grafana	10.2.0	Visualisation
Digital twin	Eclipse Ditto	3.4.3	Virtual representation of tanks
Persistence for Ditto	MongoDB	6.0	Stores things and policies
Reverse proxy	Nginx	1.25-alpine	Ditto API gateway with pre-authentication
Container platform	Docker Desktop / Compose	Latest	Orchestration of all services

CHAPTER 3
SYSTEM ANALYSIS AND DESIGN

3.1 Requirement Analysis

3.1.1 Functional Requirements

- The system shall generate pH, turbidity, temperature and water-level readings for three tanks every five seconds.
- The system shall transmit readings over MQTT on a separate topic for each tank.
- The system shall validate each reading for completeness and range, and reject invalid readings.
- The system shall classify every valid reading as SAFE, WARNING or UNSAFE using configurable thresholds.
- The system shall store every processed reading, with its status, in a time-series database.
- The system shall maintain a digital twin for each tank that reflects its latest state.
- The system shall publish an alert message whenever a tank is in WARNING or UNSAFE status.
- The system shall present live and historical data on dashboards that can be filtered by tank.

3.1.2 Non-Functional Requirements

- **Portability:** the system must run unchanged on any machine with Docker.
- **Modularity:** each component must be replaceable without affecting the others.
- **Reliability:** services must restart automatically and report their health.
- **Observability:** logs from every container must be accessible for diagnosis.
- **Configurability:** thresholds and connection settings must be adjustable without rebuilding the system.

3.1.3 Hardware and Software Requirements

Table 3.1: Hardware and software requirements

Item	Requirement
Processor	Multi-core 64-bit processor with virtualisation enabled
Memory	At least 8 GB RAM available to Docker (the Ditto Java services are memory-intensive)
Operating system	Windows 10/11, Linux or macOS with Docker Desktop
Software	Docker Desktop with Compose, PowerShell or Bash, a modern web browser
Free network ports	1883, 1880, 8086, 3000, 8085 and 9001

3.2 System Architecture

The system follows a layered, event-driven architecture, shown in Figure 3.1. Data flows from the sensor layer through the messaging layer into the edge-processing layer. From there it fans out to three destinations: persistent storage, the digital twin and the alert channel. The presentation layer reads from storage.

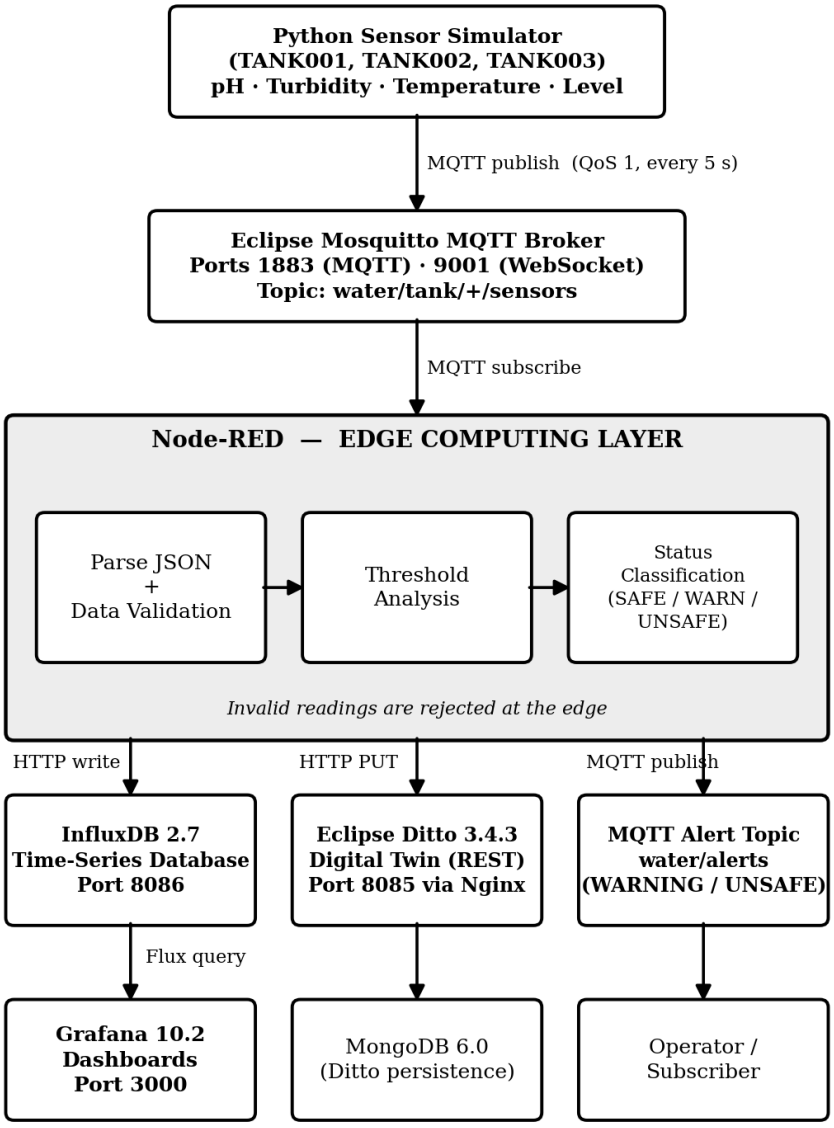


Figure 3.1: Overall system architecture

Sensor layer. The Python simulator plays the role of the physical sensors and publishes JSON readings. **Messaging layer.** Mosquitto receives all messages and delivers them to subscribers. **Edge layer.** Node-RED subscribes to every tank topic and performs validation, threshold analysis and classification. **Storage and twin layer.** InfluxDB keeps the history, and Eclipse Ditto keeps the current state. **Presentation layer.** Grafana renders dashboards, while the water/alerts topic provides push notifications for any MQTT subscriber.

Node-RED is described as the edge layer for five reasons. First, validation, analysis and classification happen locally before data reaches the database. Second, abnormal conditions are detected immediately without a database round-trip. Third, invalid readings are rejected

before storage. Fourth, alerts are produced at the source. Fifth, Node-RED bridges protocols by converting MQTT input into HTTP calls for InfluxDB and Ditto.

3.3 Data Flow

The end-to-end processing of one reading proceeds as follows:

1. The simulator creates a JSON reading for a tank and publishes it to `water/tank/<tankId>/sensors`.
2. Mosquitto accepts the message and forwards it to Node-RED, which is subscribed to `water/tank/+/sensors`.
3. Node-RED parses the JSON text into an object and validates it.
4. Valid data is analysed against the thresholds, and the overall status is calculated.
5. The processed reading is written to InfluxDB through its HTTP write API.
6. The same reading is sent to Eclipse Ditto, which updates the `waterQuality` feature of the tank's twin.
7. If the status is WARNING or UNSAFE, an alert is published to `water/alerts`.
8. Grafana queries InfluxDB every five seconds and refreshes its panels.

3.4 MQTT Topic Design and Message Format

Table 3.2: MQTT topics used by the system

Topic	Direction	Description
water/tank/TANK001/sensors	Simulator to Mosquitto	Readings of tank 1
water/tank/TANK002/sensors	Simulator to Mosquitto	Readings of tank 2
water/tank/TANK003/sensors	Simulator to Mosquitto	Readings of tank 3
water/tank/+/sensors	Mosquitto to Node-RED	Wildcard subscription for all tanks
water/alerts	Node-RED to Mosquitto	Alerts for WARNING and UNSAFE tanks

Each sensor message is a JSON document with the following structure:

```
{
  "tankId": "TANK001",
  "pH": 7.2,
  "turbidity": 2.5,
  "temperature": 28.0,
  "waterLevel": 80.0,
  "timestamp": "2026-10-03T10:30:00.000+00:00"
}
```

3.5 Docker Architecture

All containers are attached to a single user-defined bridge network, `water-network`, on which they reach each other by service name (for example `http://influxdb:8086`). Only the ports that a user needs are published to the host. Table 3.3 lists the twelve containers that make up the system.

Table 3.3: Containers in the Docker Compose deployment

Container	Image	Host port	Role
mosquitto	eclipse-mosquitto:2	1883, 9001	MQTT broker
nodered	nodered/node-red:latest	1880	Edge processing
influxdb	influxdb:2.7	8086	Time-series storage
grafana	grafana/grafana:10.2.0	3000	Dashboards
sensor-simulator	Built from python:3.11-slim	None	Sensor data generator
mongodb	mongo:6.0	Internal	Ditto persistence
ditto-policies	eclipse/ditto-policies:3.4.3	Internal	Access policies
ditto-things	eclipse/ditto-things:3.4.3	Internal	Thing management
ditto-things-search	eclipse/ditto-things-search:3.4.3	Internal	Thing search
ditto-gateway	eclipse/ditto-gateway:3.4.3	Internal	HTTP/WebSocket API
ditto-connectivity	eclipse/ditto-connectivity:3.4.3	Internal	External connections
ditto-nginx	nginx:1.25-alpine	8085	Reverse proxy with pre-authentication

Named volumes keep InfluxDB, Grafana, MongoDB and Mosquitto data across restarts, while the Node-RED flow file and the Grafana provisioning folder are bind-mounted from the project directory. Health checks are defined for the key services, and `restart: unless-stopped` keeps them running after failures.

3.6 Project Folder Structure

```
smart-water-monitoring/
|-- docker-compose.yml            All services, networks and volumes
|-- .env                          Environment variables
|-- README.md                     Project documentation
|-- mosquitto/config/
|     `-- mosquitto.conf          MQTT broker configuration
|-- simulator/
|     |-- sensor_simulator.py     Python sensor simulator
|     |-- requirements.txt        Python dependencies
|     `-- Dockerfile              Simulator container image
|-- nodered/data/
|     `-- flows.json              Edge-processing flow
|-- grafana/provisioning/
|     |-- datasources/influxdb.yml
|     `-- dashboards/
|           |-- dashboard.yml
|           `-- water-quality.json
`-- ditto/
      |-- nginx.conf              Reverse proxy for Ditto
      |-- setup-ditto.ps1         Twin setup script (Windows)
      `-- setup-ditto.sh          Twin setup script (Linux/macOS)
```

CHAPTER 4
IMPLEMENTATION

This chapter describes how each component was implemented. The screenshots were captured from the running system on 2 and 3 October 2026.

4.1 Docker Deployment

The project directory contains a single `docker-compose.yml`. The whole system is started with the command below. Figure 4.1 shows the Docker Desktop view of the running project and Figure 4.2 shows the compose output, in which all twelve containers are reported as Running or Healthy.

```
cd E:\Cloud\smart-water-monitoring
docker compose up -d
```

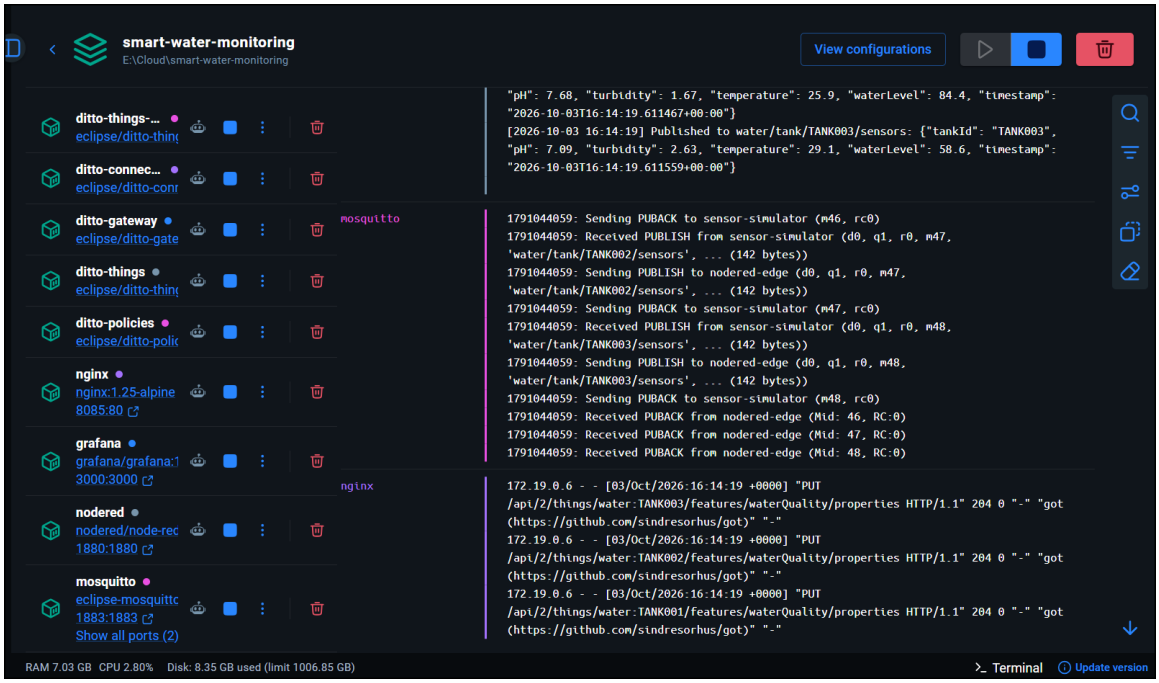


Figure 4.1: Docker Desktop showing the smart-water-monitoring containers and live logs

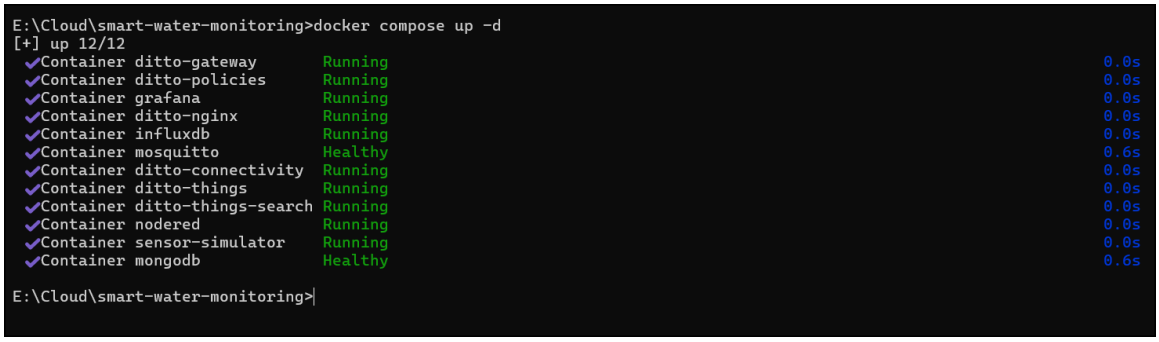


Figure 4.2: Output of docker compose up -d with all 12 containers running

The state and published ports of every service can be verified with `docker compose ps`, as shown in Figure 4.3. The Ditto services are internal to the Docker network and are reachable from the host only through the Nginx proxy on port 8085.

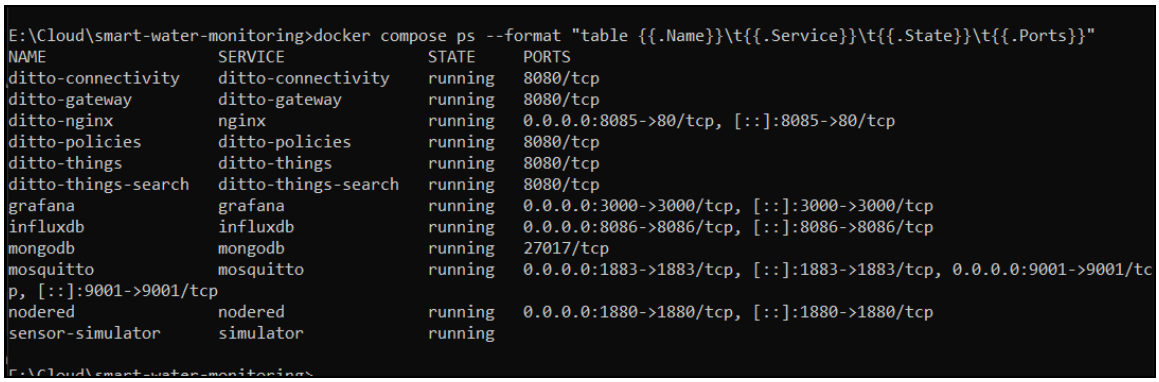


Figure 4.3: Container status and ports reported by docker compose ps

4.2 Python Sensor Simulator

The simulator, `sensor_simulator.py`, runs in its own container built from `python:3.11-slim` with the `paho-mqtt` library. It reads the broker address, port and interval from environment variables and defines a base profile for each tank. A small random variation is added to every parameter so that the values look like real sensor noise.

Table 4.1: Base profile of each simulated tank

Tank	pH	Turbidity (NTU)	Temperature (°C)	Water level (%)
TANK001	7.0	2.0	27.0	75.0
TANK002	7.5	1.5	25.0	85.0
TANK003	6.8	3.0	30.0	60.0

The random variation applied around the base values is ±0.3 for pH, ±0.5 NTU for turbidity, ±1.5 °C for temperature and ±3 % for water level. Values are clamped to physically valid ranges (pH 0 to 14, level 0 to 100 %).

To demonstrate the alert logic, the simulator injects an abnormal reading into one randomly chosen tank every sixth cycle, which is roughly every 30 seconds. The scenario is chosen at random from the list in Table 4.2.

Table 4.2: Abnormal scenarios injected by the simulator

Scenario	Parameter	Injected range	Expected class
high_pH	pH	10.0 – 12.0	UNSAFE
low_pH	pH	2.0 – 4.0	UNSAFE
high_turbidity	Turbidity	12.0 – 20.0 NTU	UNSAFE
high_temperature	Temperature	42.0 – 50.0 °C	UNSAFE
low_water_level	Water level	2.0 – 8.0 %	UNSAFE
warning_pH	pH	9.0 – 9.5	WARNING
warning_turbidity	Turbidity	6.0 – 9.0 NTU	WARNING

Each reading is published with QoS 1 to its own topic. The simulator logs every publication and marks injected readings with the tag [ABNORMAL]. Figure 4.4 shows the live log of the simulator, in which an abnormal reading for TANK002 is visible, and Figure 4.5 shows the injection of a high_temperature scenario.

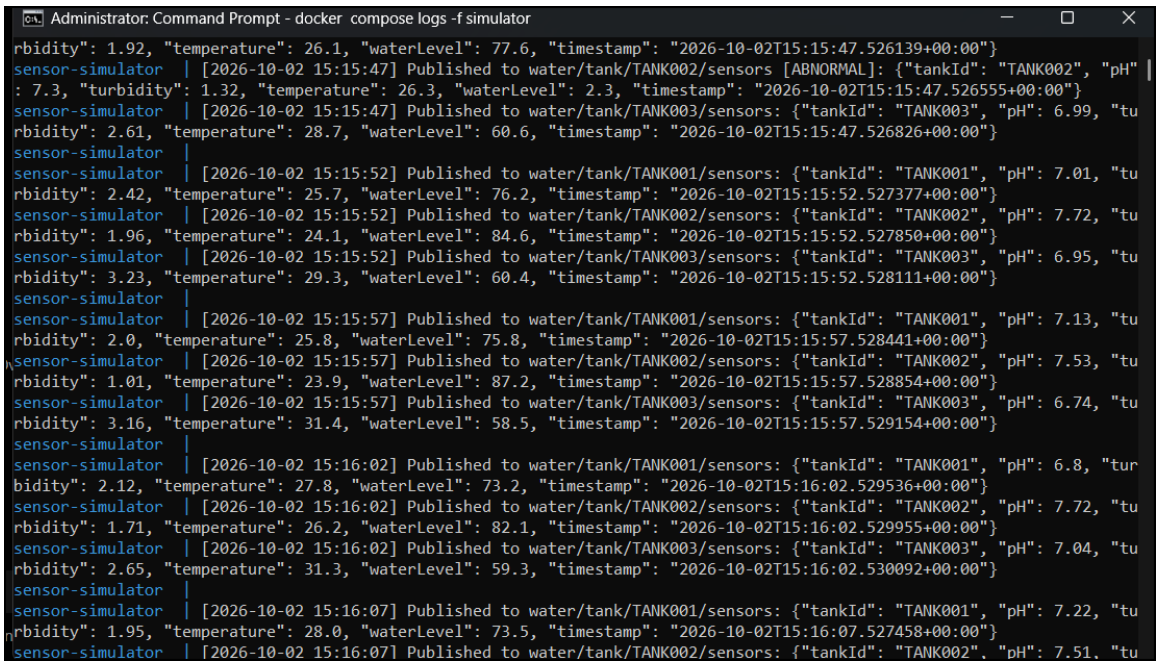


Figure 4.4: Live telemetry in the simulator log, including an [ABNORMAL] reading

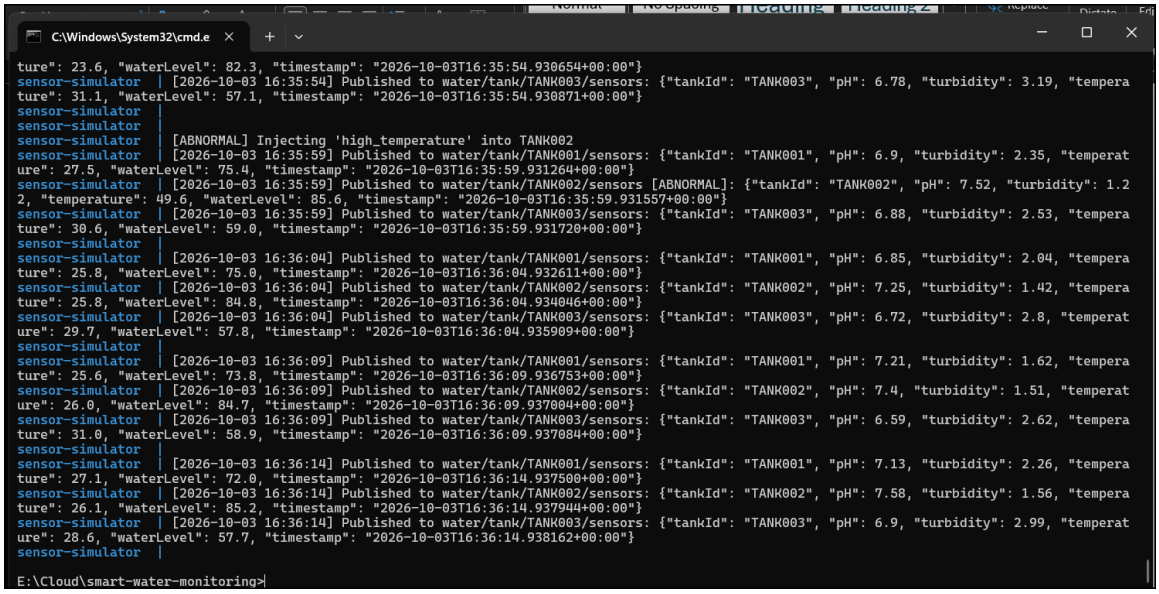


Figure 4.5: Simulator injecting a high_temperature scenario into TANK002

4.3 MQTT Broker

Mosquitto is configured with two listeners (MQTT on 1883 and WebSocket on 9001), anonymous access for the development environment, persistence enabled and all log types sent to standard output, so that traffic can be examined with `docker compose logs`. The sensor traffic can be observed directly by subscribing to the wildcard topic, as in Figure 4.6.

```
docker exec mosquitto mosquitto_sub -t "water/tank/+/sensors" -v
```

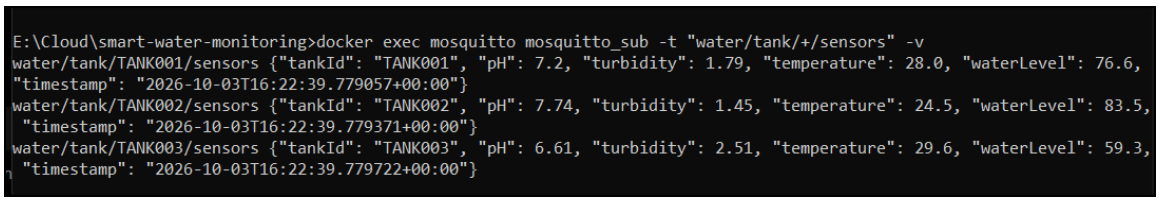


Figure 4.6: Live sensor messages received by an MQTT subscriber

The broker log in Figure 4.7 shows the complete message path: the broker receives a publication from the sensor-simulator client, delivers it to the nodered-edge client, and acknowledges the publisher. It also shows a message on the water/alerts topic that was published by Node-RED.

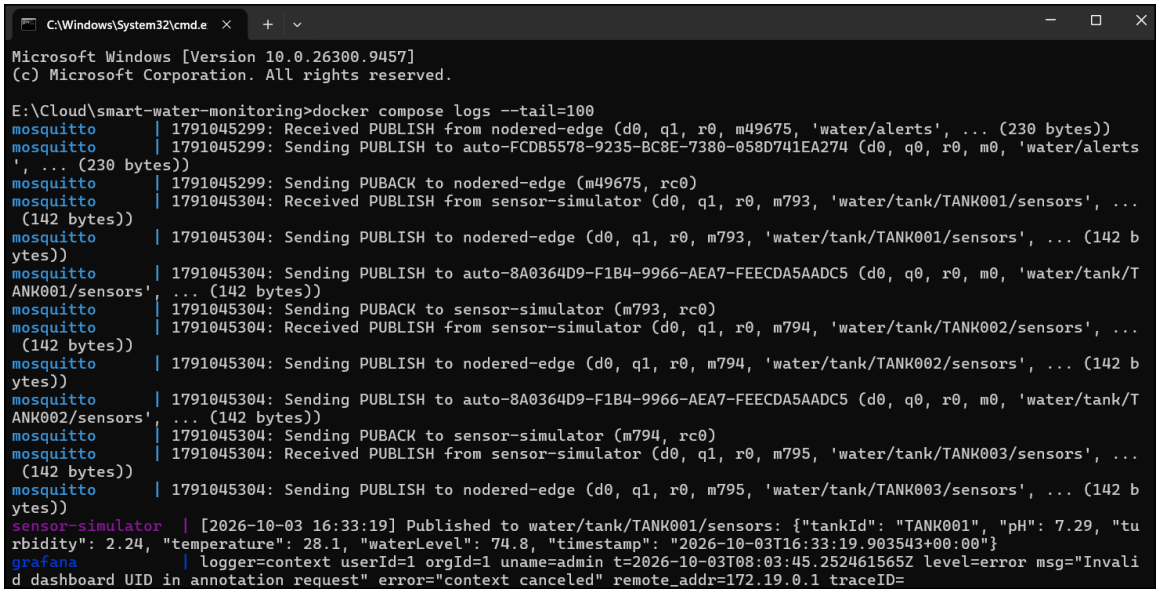


Figure 4.7: Docker Compose logs showing broker, simulator and dashboard activity

4.4 Node-RED Edge Processing Layer

The edge layer is a single Node-RED flow named *Smart Water Quality Monitoring - Edge Processing Layer*. It is stored in `flows.json` and is mounted into the container, so the flow survives container restarts. The nodes of the flow, in the order of processing, are listed below.

Table 4.3: Nodes of the Node-RED edge flow

Node	Type	Function
Water Sensor Data	mqtt in	Subscribes to water/tank/+/sensors with QoS 1
Parse JSON	json	Converts the message text to a JavaScript object
Data Validation	function	Checks required fields and value ranges; valid data on output 1, invalid on output 2
Invalid Data	debug	Displays rejected readings and the reasons
Threshold Analysis & Classification	function	Applies thresholds and computes SAFE, WARNING or UNSAFE
Prepare InfluxDB Write	function	Builds the line-protocol string and authorisation header
Write to InfluxDB	http request	POST to /api/v2/write
Prepare Ditto Update	function	Builds the PUT request for the twin's waterQuality properties
Update Ditto Twin	http request	Calls the Ditto REST API through Nginx
Alert Check	function	Creates an alert message for non-SAFE readings
Publish Alert	mqtt out	Publishes to the water/alerts topic

4.4.1 Data Validation

The validation function protects the database and the digital twin from corrupt data. A reading is rejected if any of the rules in Table 4.4 is violated; the reasons are collected and shown in the debug sidebar.

Table 4.4: Validation rules applied at the edge

Field	Rule
tankId	Must be present and a string
pH	Must be a number between 0 and 14
turbidity	Must be a number that is not negative
temperature	Must be a number between -50 and 100 °C (sensor range)
waterLevel	Must be a number between 0 and 100 %
timestamp	Must be present

4.4.2 Threshold Analysis and Classification

Thresholds are defined in one clearly marked object at the top of the function so that they can be tuned without changing the logic. Each parameter is classified separately, according to Table 4.5. The pH range of 6.5 to 8.5 and the turbidity limit of 5 NTU follow the limits commonly used for drinking water, for example in the Indian standard IS 10500:2012 and the WHO guidelines.

Table 4.5: Threshold values used for classification

Parameter	SAFE	WARNING	UNSAFE
pH	6.5 – 8.5	5.5 – 6.5 or 8.5 – 9.5	Below 5.5 or above 9.5
Turbidity (NTU)	0 – 5	5 – 10	Above 10
Temperature (°C)	20 – 35	Below 20, or 35 – 40	Above 40
Water level (%)	30 or higher	10 – 30	Below 10

The overall status of a tank follows a worst-case rule: if any parameter is UNSAFE, the tank is UNSAFE; otherwise, if any parameter is WARNING, the tank is WARNING; otherwise it is SAFE. The function also records a human-readable alert text for every parameter outside its safe range, for example "CRITICAL: Very high pH: 11.47". Figure 4.8 shows the decision logic of the complete flow.

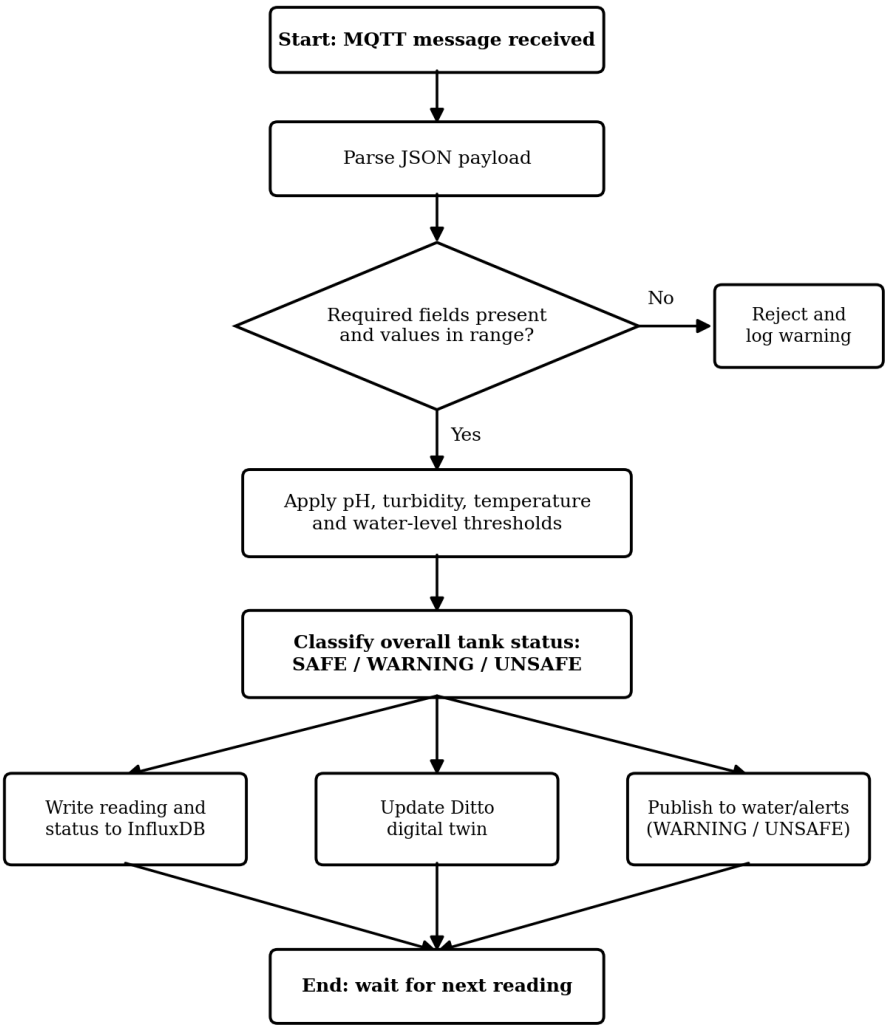


Figure 4.8: Edge-processing logic of the Node-RED flow

The processed message has the following form:

```
{
  "tankId": "TANK001",
  "pH": 7.2,
  "turbidity": 2.5,
  "temperature": 28,
  "waterLevel": 80,
  "status": "SAFE",
  "alerts": [],
  "timestamp": "2026-10-03T10:30:00.000+00:00"
}
```

Figures 4.9 and 4.10 show the flow in the Node-RED editor. The node status labels, for example "TANK003: SAFE", are updated live by the function nodes. The debug sidebar on the right lists the HTTP responses with status code 204, which confirm that writes to InfluxDB and Ditto succeeded.

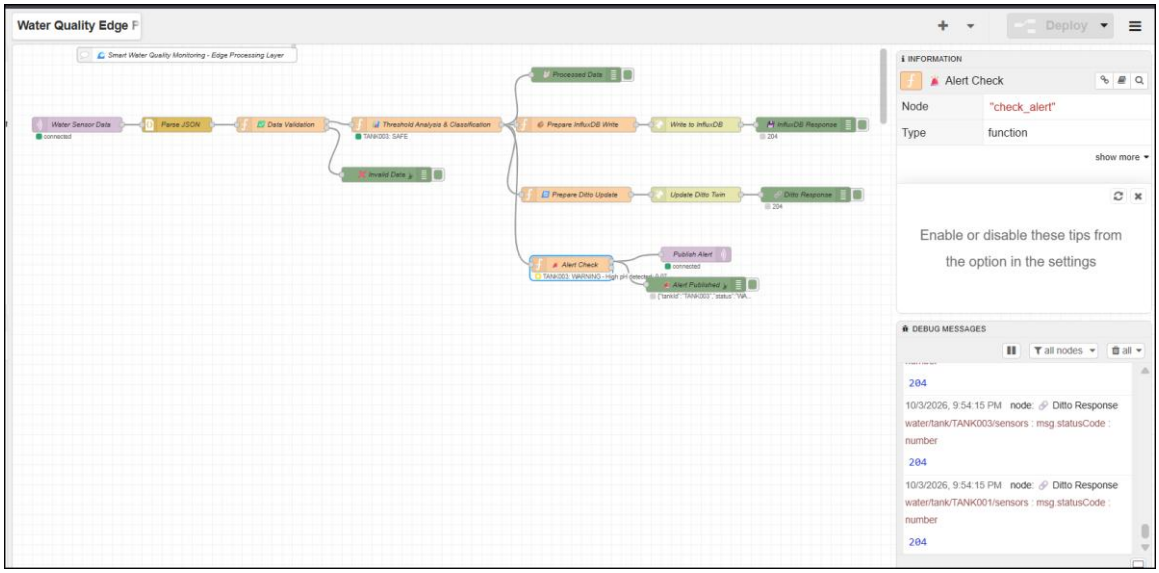


Figure 4.9: Node-RED edge flow with the Alert Check function node selected

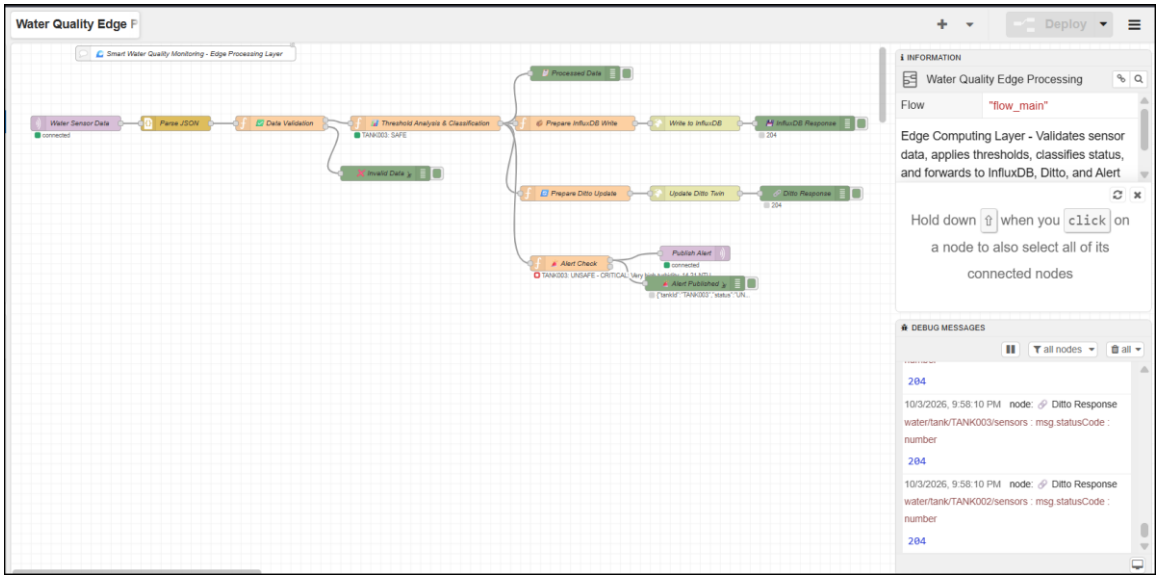


Figure 4.10: Node-RED edge flow with live node status and debug messages

4.5 InfluxDB Time-Series Storage

InfluxDB is initialised automatically at first start through environment variables that create the organisation, bucket, administrator and API token. The storage design is shown in Table 4.6.

Table 4.6: InfluxDB storage design

Item	Value
Organisation	WaterMonitoring
Bucket	water_quality
Measurement	water_quality
Tag	tankId (allows filtering by tank)
Fields	pH, turbidity, temperature, waterLevel, status, alerts
Write API	POST /api/v2/write with line protocol, millisecond precision

Node-RED converts every reading into line protocol before writing it. An example of the generated string is shown below.

```
water_quality,tankId=TANK001
pH=7.2,turbidity=2.5,temperature=28,waterLevel=80,
    status="SAFE",alerts="none"
```

The stored data can be explored in the InfluxDB Data Explorer by selecting the bucket, measurement, fields and tank, as shown in Figure 4.11. The equivalent Flux query is:

```
from(bucket: "water_quality")
  |> range(start: -1h)
  |> filter(fn: (r) => r._measurement == "water_quality")
  |> filter(fn: (r) => r.tankId == "TANK001")
  |> filter(fn: (r) => r._field == "pH")
```

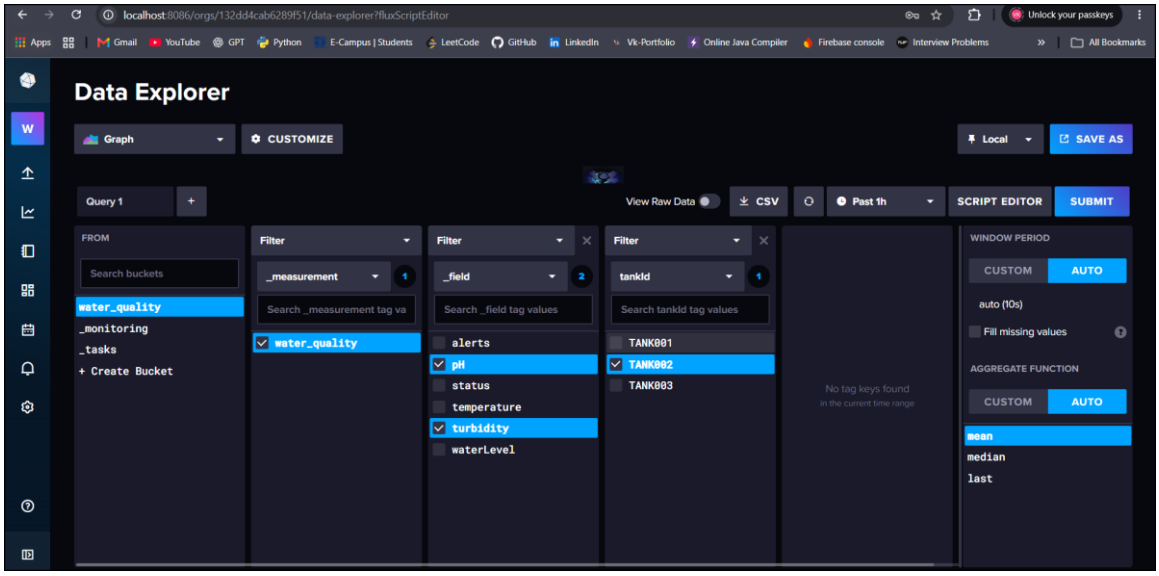


Figure 4.11: InfluxDB Data Explorer with water_quality bucket, pH and turbidity fields for TANK002

The InfluxDB container log in Figure 4.12 shows the storage engine starting, the shards being opened and the HTTP API listening on port 8086, which confirms that the database was initialised correctly.

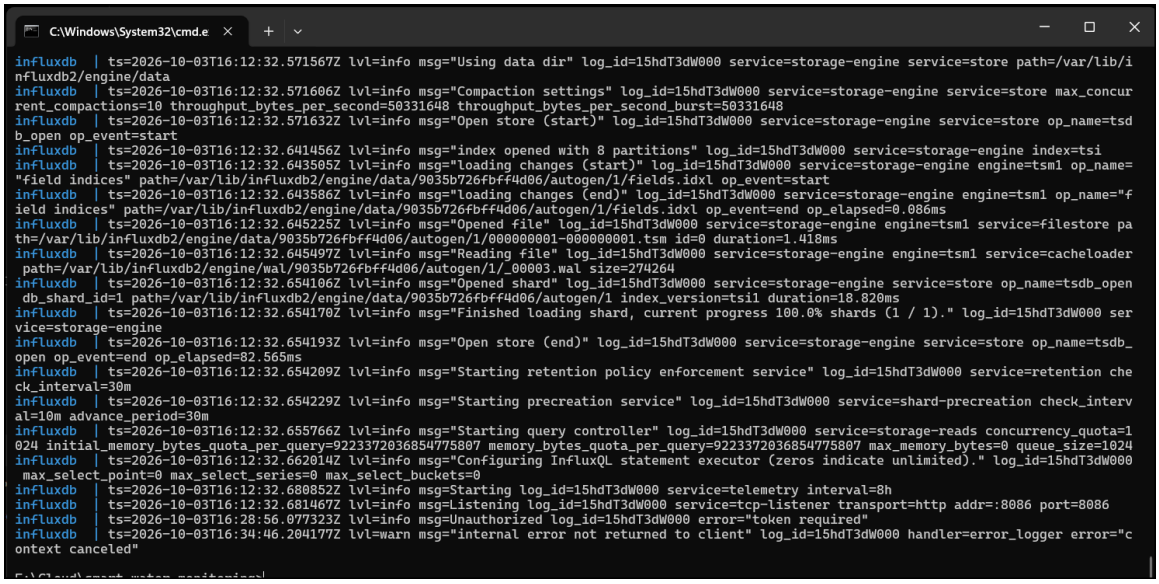


Figure 4.12: InfluxDB container start-up log

4.6 Eclipse Ditto Digital Twin

Ditto is deployed as five microservices together with MongoDB and an Nginx reverse proxy. Nginx adds the header `x-ditto-pre-authenticated: nginx:ditto` to every request, so that clients (Node-RED, curl or a browser) can use the API without a separate login in this development setup. The policy `water:tank-policy` grants this pre-authenticated subject READ and WRITE permission on all things and on the policy itself.

After Ditto becomes healthy, the script `setup-ditto.ps1` (or `setup-ditto.sh` on Linux and macOS) creates the policy and the three digital twins. Figure 4.13 shows the script waiting for Ditto, creating the policy and creating `water:TANK001`, `water:TANK002` and `water:TANK003`.

```
powershell -ExecutionPolicy Bypass -File ".\ditto\setup-ditto.ps1"
```

```
E:\Cloud\smart-water-monitoring>powershell -ExecutionPolicy Bypass -File ".\ditto\setup-ditto.ps1"
==========================================
  Eclipse Ditto - Digital Twin Setup
==========================================

[1/3] Waiting for Ditto to be ready...
  Ditto is healthy!

[2/3] Creating tank policy...
  Policy 'water:tank-policy' created successfully!

[3/3] Creating digital twins...
  Digital twin 'water:TANK001' created successfully!
  Digital twin 'water:TANK002' created successfully!
  Digital twin 'water:TANK003' created successfully!

==========================================
  Setup Complete!
==========================================

  Verify twins:
    curl http://localhost:8085/api/2/things/water:TANK001
    curl http://localhost:8085/api/2/things/water:TANK002
    curl http://localhost:8085/api/2/things/water:TANK003
```

Figure 4.13: Digital-twin setup script creating the policy and three twins

Each twin has static attributes and a dynamic `waterQuality` feature:

```
{
  "thingId": "water:TANK001",
  "policyId": "water:tank-policy",
  "attributes": {
    "tankId": "TANK001",
    "description": "Smart Water Tank TANK001",
    "location": "Plant Floor"
  },
  "features": {
    "waterQuality": {
      "properties": {
        "pH": 7.2, "turbidity": 2.5, "temperature": 28,
        "waterLevel": 80, "status": "SAFE", "alerts": [],
        "timestamp": "2026-10-03T10:30:00.000+00:00"
      }
    }
  }
}
```

The current state of any tank can be read from the REST API, for example `http://localhost:8085/api/2/things/water:TANK001`. Figure 4.14 shows the JSON returned for TANK001 in a browser, including its attributes and the latest water-quality properties with status SAFE.

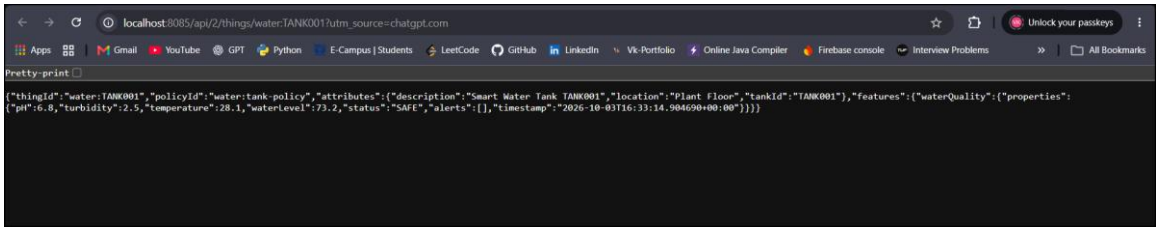


Figure 4.14: Digital twin of TANK001 returned by the Ditto REST API

The ditto-gateway log in Figure 4.15 confirms that Node-RED continuously updates the twins: each PUT request to `/api/2/things/water:TANK00x/features/waterQuality/properties` is answered with status 204 (No Content), meaning that the update was accepted.

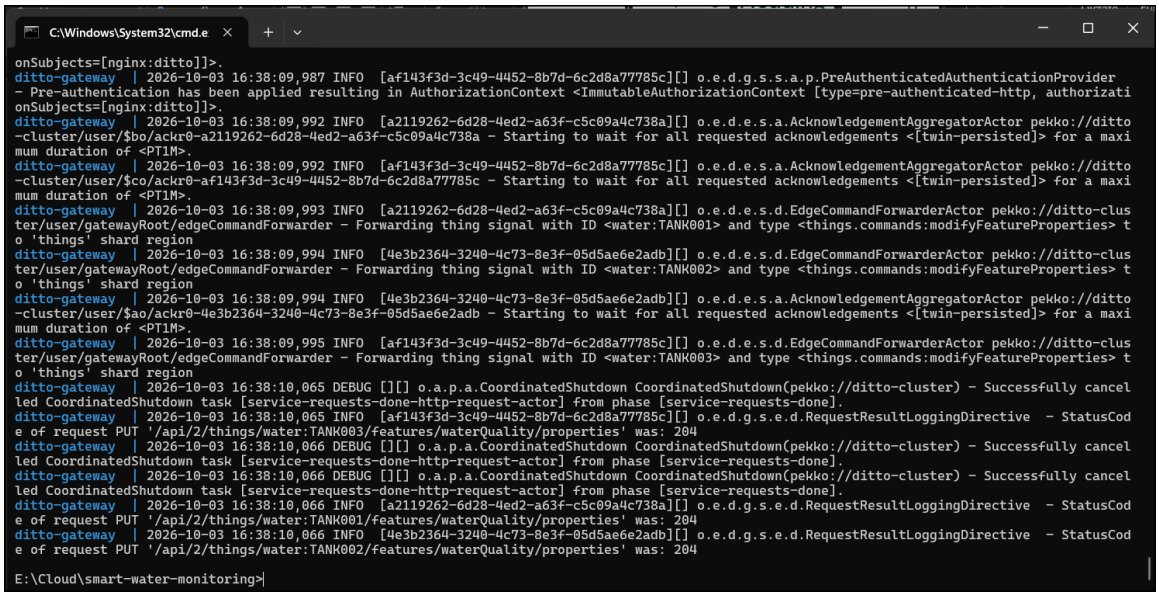


Figure 4.15: Ditto gateway log showing property updates for the twins

4.7 Grafana Dashboards

Grafana is configured entirely through provisioning files. The data source file registers InfluxDB (Flux language, organisation WaterMonitoring, default bucket water_quality), and the dashboard file loads the dashboard *Smart Water Quality Monitoring*. The dashboard refreshes every five seconds and has a *Tank ID* drop-down variable that switches all panels between TANK001, TANK002 and TANK003. It contains two rows:

- **Current Values:** stat panels for Tank Status, Current pH, Current Turbidity, Current Temperature and Water Level, each using the Flux function `last()` to display the latest reading.
- **Historical Data:** time-series graphs for pH, turbidity, temperature and water level, aggregated over a time window.

Figures 4.16, 4.17 and 4.18 show the dashboard for TANK001, TANK002 and TANK003 respectively. In each, the current values are shown at the top and the pH and turbidity history below. The narrow spikes in the history graphs correspond to the abnormal readings injected by the simulator, and they return to the normal band in the next reading.



Figure 4.16: Grafana dashboard for TANK001

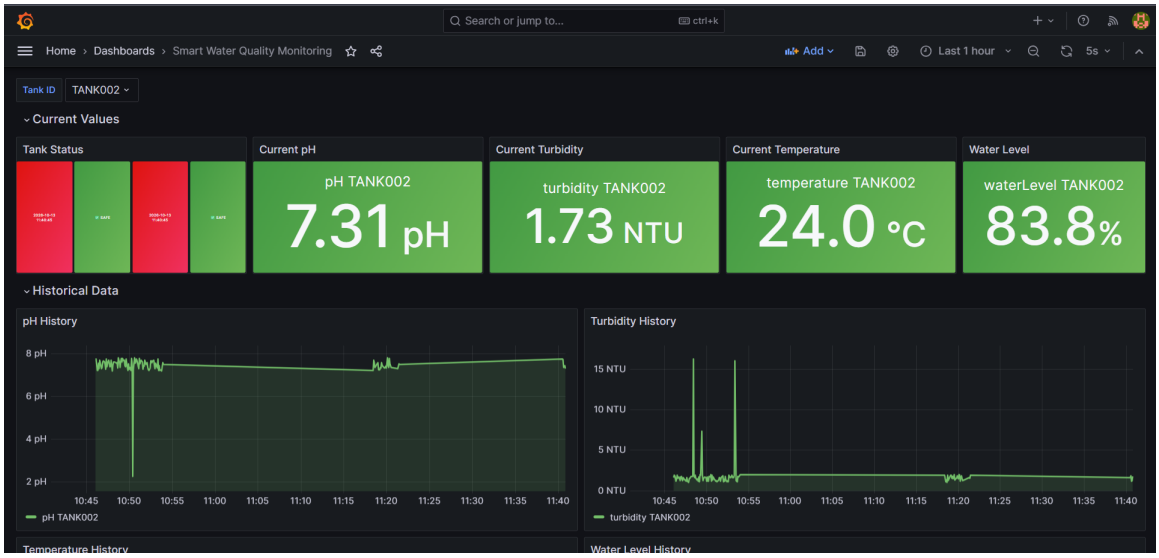


Figure 4.17: Grafana dashboard for TANK002



Figure 4.18: Grafana dashboard for TANK003

The Grafana log in Figure 4.19 shows the provisioned data source executing Flux queries against InfluxDB, which is the mechanism behind every panel.



Figure 4.19: Grafana container log showing Flux queries to InfluxDB

4.8 MQTT Alert System

When the status of a reading is WARNING or UNSAFE, the Alert Check node builds an alert document and Node-RED publishes it to `water/alerts`. Any application can subscribe to this topic to receive notifications, for example an SMS gateway, a mobile app or a control-room display. The alerts can be observed with:

```
docker exec mosquitto mosquitto_sub -t "water/alerts" -v
```

An alert for TANK002 with status UNSAFE, caused by a pH of 11.47, is shown in Figure 4.20. The message carries the tank identifier, the status, the alert text, all four readings and the timestamp.



Figure 4.20: MQTT alert received on the water/alerts topic

Figure 4.21 shows the same alert from the point of view of the broker log: Mosquitto receives a PUBLISH on `water/alerts` from the nodered-edge client and forwards it to the subscriber. Figure 4.22 shows the Node-RED log, in which alerts of both severities are listed: WARNING (for example high pH 9.15) and UNSAFE (for example very low pH, very high temperature, very low water level).

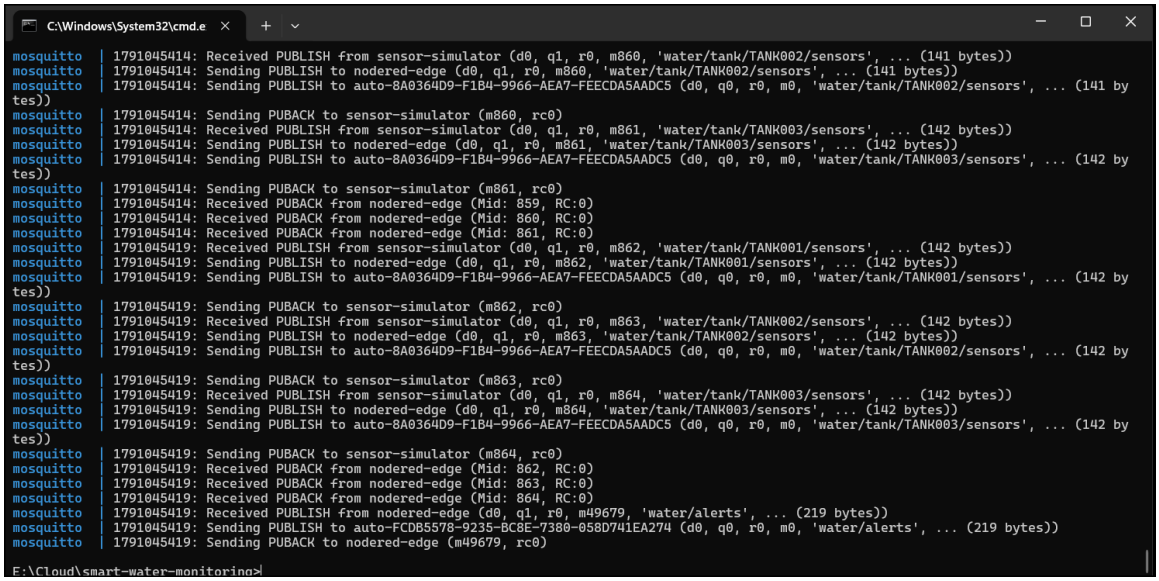


Figure 4.21: Mosquitto log showing the alert message on water/alerts

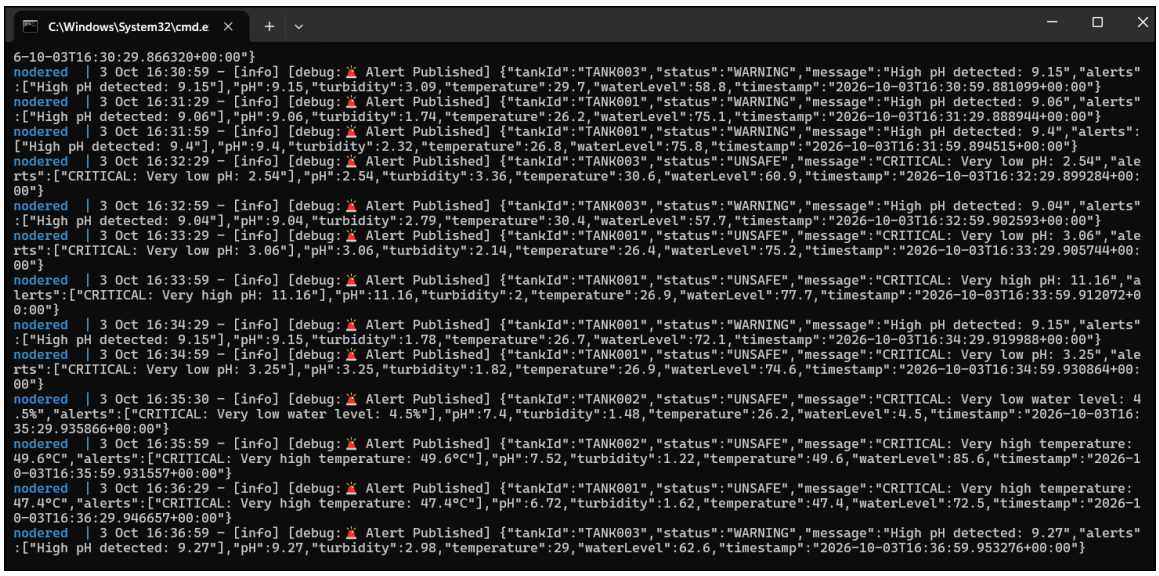


Figure 4.22: Node-RED log of alerts published for WARNING and UNSAFE tanks

CHAPTER 5
TESTING AND RESULTS

5.1 Test Environment

All tests were carried out on a Windows computer with Docker Desktop. The complete stack of twelve containers was started with `docker compose up -d`. Docker Desktop reported a memory use of about 7 GB while the system was running, which is consistent with the recommendation of at least 8 GB for Docker.

5.2 Test Cases

The test cases in Table 5.1 cover each layer and the complete pipeline. The observed behaviour was verified from the screenshots presented in Chapter 4.

Table 5.1: Test cases and results

ID	Test	Expected result	Observed result	Result
TC1	Start all services	All 12 containers Running/Healthy	docker compose up -d reports 12 of 12 started (Fig. 4.2, 4.3)	Pass
TC2	Create digital twins	Policy and 3 twins created	Script reports success for policy, TANK001, TANK002 and TANK003 (Fig. 4.13)	Pass
TC3	Simulator publishing	Three messages every 5 s	Readings for all tanks appear at 5-second intervals (Fig. 4.4)	Pass
TC4	MQTT delivery	Subscriber receives sensor data	Messages for TANK001-003 received (Fig. 4.6)	Pass
TC5	Edge processing	Status computed, writes accepted	Node-RED shows node status and HTTP 204 responses (Fig. 4.9, 4.10)	Pass
TC6	InfluxDB storage	Data queryable by tank and field	Data Explorer lists fields and tanks (Fig. 4.11)	Pass
TC7	Digital twin query	REST API returns latest state	JSON of TANK001 with status SAFE returned (Fig. 4.14)	Pass
TC8	Twin synchronisation	Property updates accepted	Gateway log shows PUT requests answered 204 (Fig. 4.15)	Pass
TC9	Dashboards	Live values and history for each tank	Three dashboards display current values and graphs (Fig. 4.16-4.18)	Pass
TC10	UNSAFE alert	Alert published on water/alerts	CRITICAL pH alert for TANK002 received (Fig. 4.20)	Pass
TC11	WARNING alert	Alert with WARNING status	High pH 9.15 alert logged (Fig. 4.22)	Pass

5.3 Results and Discussion

The system performed the complete pipeline for all three tanks without interruption. The latest values shown on the three dashboards are compared in Table 5.2. All values lie within the SAFE ranges of Table 4.5, which matches the base profile of each tank.

Table 5.2: Readings displayed on the Grafana dashboards

Tank	pH	Turbidity (NTU)	Temperature (°C)	Water level (%)
TANK001	6.85	1.57	26.1	73.3
TANK002	7.78	1.01	24.4	87.4
TANK003	7.01	3.23	29.9	57.8

The history graphs show a stable band around each tank's base value, with brief spikes whenever the simulator injected an abnormal reading. For example, the pH of the tanks rises to between 10 and 12 or drops towards 2, and the turbidity rises above 12 NTU. These excursions were classified as UNSAFE by the edge layer and produced alerts almost immediately, which demonstrates that detection happens at the edge without waiting for any dashboard query.

Each tank has its own digital twin, and the twin always reflects the most recent processed reading, including its status and alert texts. Applications can therefore ask Ditto for the present condition of a tank without knowing anything about MQTT or InfluxDB. InfluxDB, on the other hand, supplies the history that is needed for trend analysis. Using both gives the system a clear separation: the twin answers "what is the state now?", and the database answers "what happened over time?".

5.4 Observations

- End-to-end latency from publication to updated twin and stored point is short, because all components run on the same Docker network.
- Changing thresholds only requires editing one object in the Node-RED function node, with no rebuild of any container.
- Using Docker Compose makes the system reproducible: the same command starts the same system on another computer.
- The Ditto services take 60 to 90 seconds to initialise, so the twin setup script waits until the gateway reports healthy before creating the twins.

5.5 Limitations

- The readings are simulated; no physical sensors or calibration were involved.
- The Mosquitto broker allows anonymous connections, and the Ditto API is pre-authenticated through Nginx. These settings are suitable for a development setup but not for production.
- In the captured dashboards the *Tank Status* stat panel displays "No data", because Grafana's stat panel needs additional display options to show a text value. The status itself is stored in InfluxDB, available in the digital twin and used for alerts.
- The system is a single-host deployment and has no high-availability or backup configuration.

CHAPTER 6
SECURITY CONSIDERATIONS AND TROUBLESHOOTING

6.1 Security Considerations

The project is configured for convenient local development. Before any real-world deployment, the measures in Table 6.1 should be applied.

Table 6.1: Security measures recommended for production

Area	Current development setting	Recommended measure
MQTT	Anonymous access, no encryption	Enable user authentication and access-control lists; use TLS (port 8883)
InfluxDB	Default administrator password and a fixed token in the .env file	Use strong, unique secrets and store them in a secret manager; rotate tokens
Grafana	Default administrator credentials	Change the password, create role-based users, and enable HTTPS
Node-RED	Editor without authentication	Enable adminAuth and place the editor behind a firewall or VPN
Eclipse Ditto	Pre-authentication through Nginx for all requests	Use OAuth 2.0 / JWT authentication and fine-grained policies
Network	Ports published on the host	Expose only the required ports and use a reverse proxy with TLS
Secrets	.env file in the project folder	Keep the .env file out of version control

6.2 Troubleshooting

Table 6.2: Common problems and their solutions

Problem	Likely cause	Solution
A container does not start	Port already in use or too little memory	Free the port or change the mapping; allocate at least 8 GB to Docker; check with docker compose ps and docker compose logs <service>
Setup script says Ditto is not ready	The Java services need 60 to 90 seconds to initialise	Wait, then run the setup script again
Twin returns 404	Setup script was not run or failed	Run the setup script and verify the policy and twin creation messages
No sensor data	Simulator not running or broker not reachable	Check docker compose logs simulator and subscribe to water/tank/+/sensors
Grafana panels are empty	Time range too short or data source unreachable	Select Last 1 hour, confirm the InfluxDB data source and the tank selection
Node-RED shows no activity	Flow not deployed or MQTT broker disconnected	Open the editor, click Deploy and check that the MQTT node shows connected

CHAPTER 7
FUTURE ENHANCEMENTS

The modular design makes several extensions possible without redesigning the system:

- **Real sensors:** replace the simulator with ESP32 or Arduino devices using pH, turbidity, temperature and ultrasonic level probes that publish the same JSON message.
- **Secure communication:** enable TLS, authentication and access control for MQTT, and OAuth-based security for Ditto and Grafana.
- **Notifications:** connect the water/alerts topic or Grafana alert rules to e-mail, SMS, Telegram or a mobile application.
- **Machine learning:** apply anomaly detection and forecasting to the stored history to predict contamination before thresholds are crossed.
- **Actuation:** use desired properties of the digital twin to control pumps, valves or chlorine dosing automatically.
- **Scalability:** deploy on Kubernetes or a cloud platform, use MQTT clustering, and monitor many tanks and sites.
- **Direct Ditto connectivity:** use the Ditto connectivity service to receive MQTT messages directly instead of HTTP calls from Node-RED.
- **Reporting:** generate periodic compliance reports from InfluxDB data.

CHAPTER 8
CONCLUSION

This project successfully designed and implemented a complete IoT platform for monitoring the water quality of storage tanks and for maintaining their digital twins. Simulated sensors publish pH, turbidity, temperature and water-level readings through an MQTT broker to a Node-RED edge layer, which validates each reading, classifies it as SAFE, WARNING or UNSAFE, and distributes the result to InfluxDB for storage, to Eclipse Ditto for the digital twin, and to an MQTT alert topic. Grafana dashboards provide live and historical views for all three tanks.

The testing showed that all twelve containers start from a single Docker Compose command, that data flows correctly through every layer, and that abnormal conditions are detected and announced within moments of being published. The use of open-source components, standard protocols and containers makes the solution inexpensive, portable and easy to extend. With the addition of physical sensors, secure communication and predictive analytics, the same architecture can serve as the basis of a real-world smart water-management system.

REFERENCES

[1] Eclipse Foundation, "Eclipse Ditto Documentation." [Online]. Available: https://eclipse.dev/ditto/

[2] Eclipse Foundation, "Eclipse Mosquitto: An open source MQTT broker." [Online]. Available: https://mosquitto.org/

[3] OASIS, "MQTT Version 3.1.1 OASIS Standard," 2014. [Online]. Available: https://mqtt.org/

[4] OpenJS Foundation, "Node-RED Documentation." [Online]. Available: https://nodered.org/docs/

[5] InfluxData, "InfluxDB OSS v2 Documentation." [Online]. Available: https://docs.influxdata.com/influxdb/v2/

[6] Grafana Labs, "Grafana Documentation." [Online]. Available: https://grafana.com/docs/grafana/latest/

[7] Docker Inc., "Docker and Docker Compose Documentation." [Online]. Available: https://docs.docker.com/

[8] M. Grieves, "Digital Twin: Manufacturing Excellence through Virtual Factory Replication," White Paper, 2014.

[9] W. Shi, J. Cao, Q. Zhang, Y. Li and L. Xu, "Edge Computing: Vision and Challenges," IEEE Internet of Things Journal, vol. 3, no. 5, pp. 637-646, 2016.

[10] World Health Organization, "Guidelines for Drinking-water Quality, 4th edition incorporating the first addendum," Geneva, 2017.

[11] Bureau of Indian Standards, "IS 10500:2012 Drinking Water - Specification (Second Revision)," New Delhi, 2012.

APPENDIX A: SERVICE URLS AND PORTS

Table 8.1: Service endpoints (local deployment)

Service	URL	Access
Grafana dashboard	http://localhost:3000	Administrator login (see .env file)
Node-RED editor	http://localhost:1880	No authentication in development
InfluxDB UI	http://localhost:8086	Administrator login (see .env file)
Ditto API, all things	http://localhost:8085/api/2/things	Pre-authenticated through Nginx
Ditto API, TANK001	http://localhost:8085/api/2/things/water:TANK001	Pre-authenticated through Nginx
Ditto API, TANK002	http://localhost:8085/api/2/things/water:TANK002	Pre-authenticated through Nginx
Ditto API, TANK003	http://localhost:8085/api/2/things/water:TANK003	Pre-authenticated through Nginx
MQTT broker	localhost:1883 (WebSocket 9001)	Anonymous in development

APPENDIX B: OPERATIONAL COMMANDS

```
# Start every service
docker compose up -d

# Create the digital twins (after ~90 seconds)
powershell -ExecutionPolicy Bypass -File ".\ditto\setup-ditto.ps1"

# Show container status
docker compose ps

# Follow simulator output
docker compose logs -f simulator

# Subscribe to all sensor data
docker exec mosquitto mosquitto_sub -t "water/tank/+/sensors" -v

# Subscribe to alerts
docker exec mosquitto mosquitto_sub -t "water/alerts" -v

# Publish a test reading that should raise an UNSAFE alert
docker exec mosquitto mosquitto_pub \
  -t "water/tank/TANK001/sensors" \
  -m '{"tankId":"TANK001","pH":3.0,"turbidity":18.5,
       "temperature":48.0,"waterLevel":5.0,
       "timestamp":"2026-10-03T12:00:00Z"}'

# Query the digital twin
curl http://localhost:8085/api/2/things/water:TANK001

# Stop all services
docker compose down
```

APPENDIX C: SOURCE CODE EXTRACTS

C.1 Reading generation in the simulator (sensor_simulator.py)

```
def generate_sensor_data(tank_id, tank_config, abnormal=None):
    c = tank_config
    pH = round(c['pH_base'] + random.uniform(-0.3, 0.3), 2)
    turbidity = round(c['turbidity_base'] + random.uniform(-0.5, 0.5), 2)
    temperature = round(c['temperature_base'] + random.uniform(-1.5, 1.5), 1)
    waterLevel = round(c['waterLevel_base'] + random.uniform(-3.0, 3.0), 1)

    if abnormal:                    # inject an abnormal scenario
        low, high = abnormal['range']
        value = round(random.uniform(low, high), 2)
        # ... the selected parameter is replaced by 'value'

    return {'tankId': tank_id, 'pH': pH, 'turbidity': turbidity,
            'temperature': temperature, 'waterLevel': waterLevel,
            'timestamp': datetime.now(timezone.utc).isoformat()}
```

C.2 Thresholds and status classification (Node-RED function node)

```
var THRESHOLDS = {
    pH:          { SAFE_MIN: 6.5, SAFE_MAX: 8.5,
                   WARN_MIN: 5.5, WARN_MAX: 9.5 },
    turbidity:   { SAFE_MAX: 5, WARN_MAX: 10 },
    temperature: { SAFE_MIN: 20, SAFE_MAX: 35, WARN_MAX: 40 },
    waterLevel:  { SAFE_MIN: 30, WARN_MIN: 10 }
};

// ... each parameter is classified SAFE, WARNING or UNSAFE ...

var overallStatus;
if (paramStatuses.includes('UNSAFE'))
    overallStatus = 'UNSAFE';
else if (paramStatuses.includes('WARNING'))
    overallStatus = 'WARNING';
else
    overallStatus = 'SAFE';
```

C.3 Digital-twin update (Node-RED function node)

```
msg.url = dittoUrl + '/things/water:' + data.tankId
        + '/features/waterQuality/properties';
msg.method = 'PUT';
msg.headers = { 'Content-Type': 'application/json' };
msg.payload = { pH: data.pH, turbidity: data.turbidity,
                temperature: data.temperature,
                waterLevel: data.waterLevel, status: data.status,
                alerts: data.alerts, timestamp: data.timestamp };
```

APPENDIX D: EDGE LAYER VARIANTS – NODE-RED AND ECLIPSE IOFOG EDITIONS

D.1 The Shared Core Concept (Both Folders)

Both folders solve the exact same real-world problem: IoT-based Industrial Water Tank Monitoring and Digital Twin.

- **3 Physical Tanks:** Simulates three industrial tanks (TANK001, TANK002, TANK003).
- **Telemetry Generated (every 5 s):** pH (0 – 14), Turbidity (0 – 20 NTU), Temperature (20 – 50 °C) and Water Level (0 – 100 %).
- **Periodic Fault Scenarios:** Every 30 seconds, an anomaly is injected (e.g. acidic pH 2.6, high turbidity 15 NTU, low water level 5 %).
- **Cloud Storage:** InfluxDB stores all time-series historical data.
- **Visualization:** Grafana (localhost:3000) displays live gauges and charts.
- **Digital Twin:** Eclipse Ditto (localhost:8085) maintains the live virtual state of each tank.

D.2 The Core Difference: How the Edge Computing Layer Works

The only difference between the two folders is **where** and **how** the Edge Computing is processed:

```
[Sensors] --> [Mosquitto MQTT] --> [EDGE COMPUTING LAYER]
                                            |
                                            v
                          [InfluxDB + Ditto + Alerts]

The EDGE COMPUTING LAYER is implemented differently in each folder:

  FOLDER 1 (Node-RED)            FOLDER 2 (Eclipse ioFog)
  Visual Flow Canvas             Containerized Microservice
  (Port 1880)                    (iofog-edge-processor)
```

D.3 Folder 1: smart-water-monitoring (Node-RED Edge Edition)

In this folder, the edge layer is Node-RED, a visual, low-code edge workflow engine.

Edge pipeline

- **Input:** MQTT In node listens to topic water/tank/+/sensors.
- **Validation Node:** A JavaScript function checks that fields are present and within valid ranges.

- **Threshold and Classification Node:** Applies quality thresholds and assigns SAFE, WARNING or UNSAFE.
- **3 Output Branches:** Branch 1: line protocol formatter, then HTTP POST to InfluxDB. Branch 2: JSON payload formatter, then HTTP PUT to Eclipse Ditto. Branch 3: alert checker, then MQTT Out node to water/alerts (triggered on WARNING or UNSAFE).

How to view and demonstrate

- Open browser: http://localhost:1880
- The visual graph of nodes and wires can be shown to the evaluator.
- Click the Debug icon on the top right of the editor to see live parsed payloads.

D.4 Folder 2: smart-water-monitoring-iofog (Eclipse ioFog Edge Edition)

In this folder, Node-RED is removed. Instead, the edge layer is an Eclipse ioFog Edge Microservice container (iofog-edge-processor).

Edge pipeline

- **Microservice Code:** Implemented in Python (iofog_edge_processor.py).
- **Architecture:** Follows the Eclipse ioFog ECN (Edge Compute Network) model defined in ecn.yaml.
- **Processing:** Connects as an autonomous edge daemon to Mosquitto; executes local data validation and threshold classification rules in microsecond latency; pushes clean data to InfluxDB and updates Eclipse Ditto via REST API; publishes instant alerts to water/alerts.

How to view and demonstrate

Open Command Prompt and run:

```
cd E:\Cloud\smart-water-monitoring-iofog
docker compose logs -f iofog-edge-processor
```

This shows the live streaming edge processor terminal logs with status markers for SAFE, WARNING and UNSAFE readings, as captured in Figure D.1.

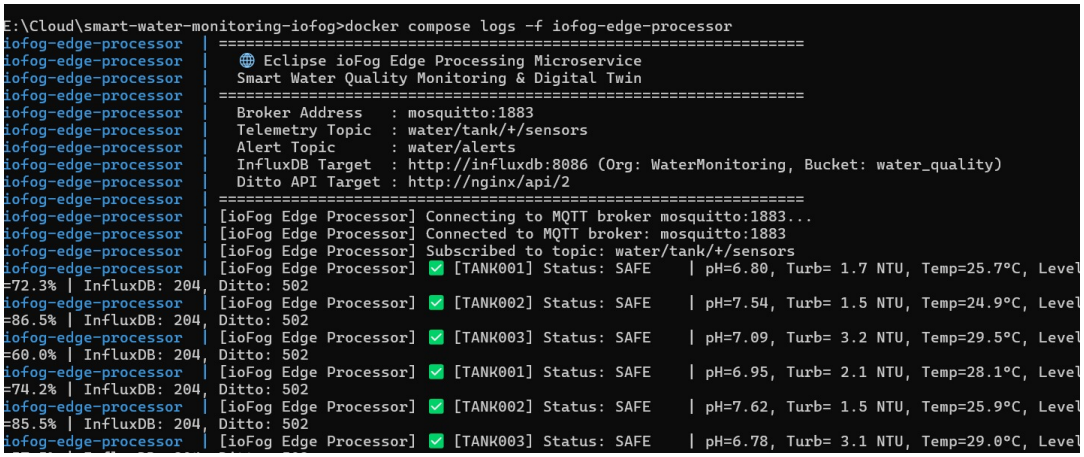


Figure D.1: Live logs of the Eclipse ioFog edge processor microservice (iofog-edge-processor)

D.5 Summary Comparison for Viva / Presentation

Table D.1: Comparison of the Node-RED and Eclipse ioFog edge editions

Feature	Folder 1: smart-water-monitoring	Folder 2: smart-water-monitoring-iofog
Edge Paradigm	Low-Code / Visual Edge Workflow	Containerized Edge Microservice / ioFog ECN
Technology	Node-RED (Node.js runtime)	Eclipse ioFog Python Microservice
Edge URL / Interface	Web GUI: http://localhost:1880	Terminal / Docker Logs / iofogctl CLI
Strength to Highlight	"Easy to visually reconfigure thresholds and business logic on the fly."	"Lightweight, headless, enterprise edge microservice managed via ioFog orchestration."
How to Start	cd E:\Cloud\smart-water-monitoring, then .\start.bat	cd E:\Cloud\smart-water-monitoring-iofog, then .\start.bat
Dashboard	http://localhost:3000 (Grafana)	http://localhost:3000 (Grafana)
Digital Twin	http://localhost:8085 (Ditto)	http://localhost:8085 (Ditto)